DA3452 – Speech and Language Technology

Module Test 1 – Preparation Guide

Representing Language Computationally

What the test covers. Module Test 1 covers **classroom material from Module 1**, the **required readings**, and the **ideas and procedures in the two reading/coding assignments**. The goal is not only to recall definitions, but to reason through small examples and calculations.

Required reading boundary

Jurafsky–Martin, Chapter 2

- §2.1 – Words
- §2.2 – Morphemes: Parts of Words
- §2.3 – Unicode
- §2.4 – Subword Tokenization: Byte-Pair Encoding
- §2.5 – Corpora

Jurafsky–Martin, Chapter 11

- §11.1.1 – Representing Documents as Vectors
- §11.1.2 – Term Weighting, **through TF–IDF**

Do not continue to §11.1.3 for this test.

What you should be able to do without notes

- **Distinguish** word, morpheme, character, token, code point, glyph, and encoded byte.
- **Explain** why words are not universal computational units and why unseen words remain a problem.
- **Trace** the path abstract character → code point → UTF-8 bytes.
- **Determine** how many UTF-8 bytes a code point requires and manually encode a simple example.
- **Inspect** canonically equivalent strings and explain what NFC changes computationally.
- **Run by hand** a few BPE merges, update the vocabulary, and apply learned merge rules to an unseen word.
- **Explain** why BPE subwords need not be morphemes and how byte-level BPE guarantees coverage.
- **Construct** a shared feature vocabulary, a Bag-of-Words vector, and a small document–term matrix.
- **Recognize** sparsity and explain what sparse storage changes – and what it does not change.
- **Compute** log-weighted TF, DF, IDF, and TF–IDF for a small corpus.
- **Predict** how TF–IDF weights change when document frequency or the corpus changes.
- **Reason** about failures caused by lost word order, synonymy, polysemy, and context/composition.

Also examinable from lecture

- Canonical equivalence and NFC normalization.
- Unicode normalization vs task-specific normalization.
- Tokenizer **training** vs using a fixed tokenizer to **encode** new text.
- Character-level vs byte-level BPE at a conceptual level.
- Vocabulary size vs sequence length; token fertility and why it matters.
- Sparsity in document vectors and sparse storage as an efficiency choice.
- The course convention for TF, DF, IDF, and TF–IDF.
- What BoW/TF–IDF preserve, collapse, and discard.

Coding-assignment ideas you should understand

- How your UTF-32/UTF-8 encoder was checked against Python.
- What `ord()`, `encode("utf-8")`, and Unicode normalization return or change.
- The logic of a small BPE trainer/encoder: count pairs, merge, record order, then apply fixed rules.
- How to construct BoW/TF–IDF values from counts rather than relying only on a library call.
- How changing the corpus can change DF, IDF, and therefore document weights.

Boundary to remember. The n-gram language-model material begun in Lecture 6 belongs to **Module 2** and is **not part of Module Test 1**. For Module 1, stop at representing text as weighted document vectors and reasoning about the choices and limitations that lead there.